

Module	Task	Assessment type
Artificial Intelligence and Machine Learning	3	Text classification

Tutor: Mr. Shiv Kumar Yadav

Name: Dipendra Roka

Group: L6CG18

Canvas ID:2418541

Submission Date: 2026/5/10

Github link:

Abstract

This report presents the work done for final assessment, where the task was to build deep learning models to classify tweets as either hate speech (racist or sexist) or non-hate speech. The dataset used was a Kaggle dataset containing 31,962 labelled tweets, out of which only about 7% were hate speech. This class imbalance made the problem quite challenging since a model that always predicts "non-hate" would still get around 93% accuracy without learning anything useful.

Three models were built and compared: a Simple RNN, a Bidirectional LSTM, and a Bidirectional LSTM using pretrained GloVe Twitter embeddings. All models used class weighting and threshold tuning on the validation set to deal with the imbalance, and the key metric used was macro F1 rather than plain accuracy.

The Simple RNN achieved the best result overall, with a macro F1 of 0.8306 on the test set. The Bidirectional LSTM was very close at 0.8281, while the GloVe-based model reached 0.7954. Interestingly, the frozen GloVe embeddings did not outperform the trainable embeddings on this relatively small dataset. The project showed that even a simple RNN with proper regularisation and threshold tuning can perform well on imbalanced text classification tasks.

Table of Contents

1. Introduction	5
2. Dataset Description	6
3. Data Visualisation	8
3.1 Class Distribution	8
3.2 Tweet Length Distribution	8
3.3 Word Clouds	9
3.4 Top 20 Most Frequent Words	10
4. Model 1 – Simple RNN	11
4.1 Architecture	11
4.2 Training Curves	11
4.3 Confusion Matrix	12
5. Model 2 – Bidirectional LSTM.....	14
5.1 Architecture	14
5.2 Training Curves	14
5.3 Confusion Matrix	15
6. Model 3 – Bidirectional LSTM with GloVe Embeddings	17
6.1 What are GloVe Embeddings?	17
6.2 Training Curves	17
6.3 Confusion Matrix	17
7. Overall Model Comparison.....	19
7.1 Validation Accuracy Over Epochs	19
7.2 Final Macro F1 Comparison	19
8. Threshold Tuning.....	21
9. Error Analysis	22
10. Challenges Faced	23
11. Conclusion	24

Table of figures

Figure 1	Class distribution showing heavy imbalance (93% non-hate vs 7% hate)	8
Figure 2	Distribution of tweet lengths (in words) before cleaning	8
Figure 3	Word cloud of all tweets after cleaning	9
Figure 4	Word clouds by class – Non-Hate (left) and Hate (right)	9
Figure 5	Top 20 most frequent words in the dataset after cleaning	10
Figure 6	Simple RNN training and validation accuracy/loss over epochs	12
Figure 7	Confusion matrix for Simple RNN on test set (threshold = 0.894)	12
Figure 8	BiLSTM training and validation accuracy/loss over epochs	15
Figure 9	Confusion matrix for BiLSTM on test set (threshold = 0.807)	15
Figure 10	BiLSTM + GloVe training and validation accuracy/loss over epochs	17
Figure 11	Confusion matrix for BiLSTM + GloVe on test set (threshold = 0.843)	18
Figure 12	Validation accuracy for all three models over training epochs	19
Figure 13	Macro F1 scores on test set for all three models	19

1. Introduction

Text classification is the process of automatically assigning categories to pieces of text. One important application of this is detecting hate speech on social media platforms like Twitter. With millions of tweets posted every day, it is not possible for human moderators to review all content manually, so automated systems are needed.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs) are popular choices for text classification because they can process sequences of words and understand the context in a sentence. Unlike simple bag-of-words models, these networks remember what came before in the text, which helps understand meaning better.

The objective of this project was to build and compare three sequence-based deep learning models for binary classification of tweets as racist/sexist ("hate") or non-hate. The project also explored the effect of using pretrained word embeddings (GloVe trained on Twitter data) versus learning embeddings from scratch. A key challenge was the severe class imbalance in the dataset, which required special handling to avoid models that just predict the majority class.

2. Dataset Description

The dataset is a tweet hate-speech dataset from Kaggle, originally used for a text classification challenge. It contains tweets that have been labelled as either racist/sexist hate speech (label = 1) or non-hate (label = 0). The test CSV provided by Kaggle did not have labels, so only the training CSV was used and then split manually into train, validation, and test sets.

Item	Value
Total Tweets in Dataset	31,962
Total Classes	2 (Non-Hate, Hate)
Non-Hate (Class 0)	29,720 (92.99%)
Hate (Class 1)	2,242 (7.01%)
Imbalance Ratio	13.3 : 1
Training Set	25,539 tweets (80%)
Validation Set	3,189 tweets (10%)
Test Set	3,193 tweets (10%)
Vocabulary Size	20,000 (top words)
Sequence Length (95th pct)	13 tokens

The split was done using stratified sampling so each split kept the same 7% hate ratio. The tokeniser was only fitted on the training set to prevent data leakage into validation and test.

Text Cleaning Pipeline

Each tweet was cleaned using the following steps:

1. Expand contractions (e.g. "don't" becomes "do not")
2. Lowercase all text
3. Remove URLs, @mentions, hashtag symbols, numbers, and non-ASCII characters
4. Tokenise, remove stopwords, and lemmatise
5. Drop consecutive duplicate words

After cleaning, 41 tweets became empty and were dropped, leaving 31,921 usable tweets.

3. Data Visualisation

3.1 Class Distribution

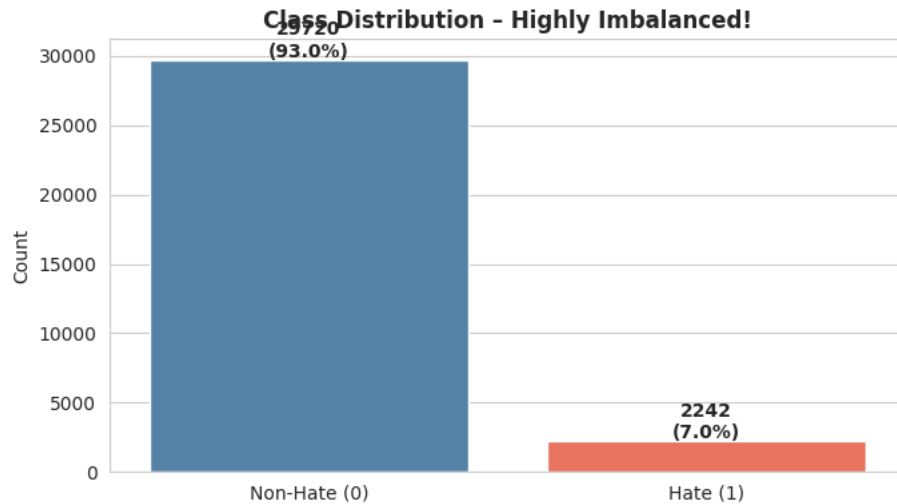


Figure 1 Class distribution showing heavy imbalance (93% non-hate vs 7% hate)

The dataset is very heavily imbalanced. Nearly 93% of tweets are non-hate speech, and only 7% are hate. This means standard accuracy is not a reliable metric here. A model that just predicts "non-hate" for everything would still score 93% accuracy. For this reason, macro F1 was used as the primary evaluation metric.

3.2 Tweet Length Distribution

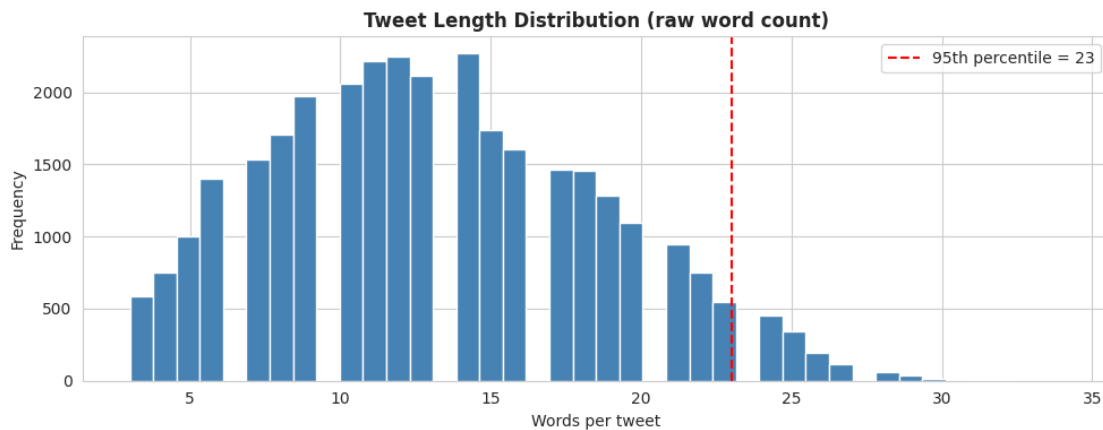


Figure 2 Distribution of tweet lengths (in words) before cleaning

more aggressive and targeted language. This confirms that the content of the words does carry signal for classification.

3.4 Top 20 Most Frequent Words

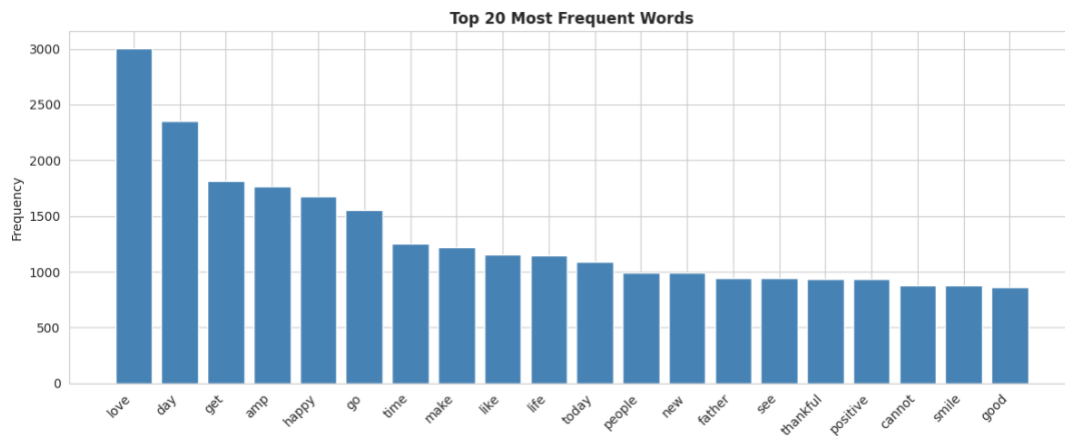


Figure 5 Top 20 most frequent words in the dataset after cleaning

The most frequent words are general Twitter terms like "amp" (from "&"), "love", and "say". These top words give a general sense of the kind of language used in the dataset.

4. Model 1 – Simple RNN

4.1 Architecture

The Simple RNN was intentionally kept straightforward as a baseline model. Here is the architecture used:

Layer	Details	Purpose
Embedding	20000 vocab, 100 dim	Word to vector
SpatialDropout1D	Rate = 0.4	Prevent overfitting in embedding
SimpleRNN	64 units, dropout 0.4	Sequence learning
Dense	32 units, ReLU, L2 reg	Feature extraction
Dropout	Rate = 0.5	Prevent overfitting
Dense (output)	1 unit, Sigmoid	Binary classification

Total parameters: 2,012,673 (all trainable). Optimizer: Adam (lr = 0.001). Loss: Binary Crossentropy. The model was trained with class weights (0.54 for non-hate, 7.13 for hate) to handle the imbalance.

4.2 Training Curves

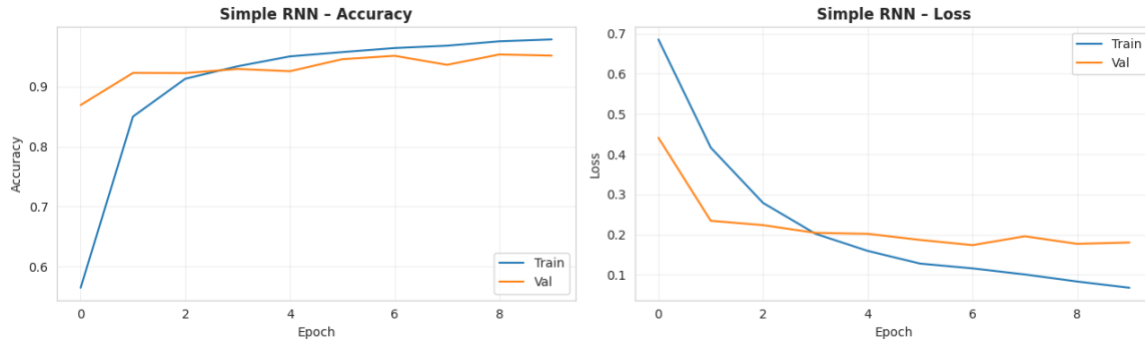


Figure 6 Simple RNN training and validation accuracy/loss over epochs

The RNN trained for 10 epochs before early stopping triggered. The training accuracy kept increasing but validation accuracy plateaued, suggesting a slight overfit. Early stopping with `restore_best_weights` ensured the best validation weights were used. Training took only 0.53 minutes (about 32 seconds), making it by far the fastest model.

4.3 Confusion Matrix

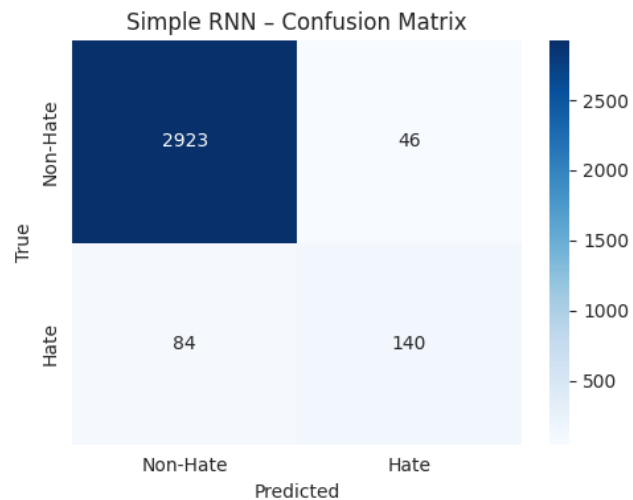


Figure 7 Confusion matrix for Simple RNN on test set (threshold = 0.894)

Metric	Value
Accuracy	0.9593
Precision (hate)	0.7527
Recall (hate)	0.6250
F1 (hate)	0.6829
Macro F1	0.8306
Best Threshold	0.894
Training Time	0.53 min

The Simple RNN achieved the best macro F1 of all three models at 0.8306. The threshold was tuned to 0.894, meaning the model needed to be quite confident before predicting "hate". This high threshold helped reduce false positives.

5. Model 2 – Bidirectional LSTM

5.1 Architecture

The Bidirectional LSTM was expected to outperform the Simple RNN because it reads the tweet in both forward and backward directions, giving the model better context. It also added Batch Normalization after the dense layer.

Layer	Details	Purpose
Embedding	20000 vocab, 100 dim	Word to vector
SpatialDropout1D	Rate = 0.3	Embedding regularisation
Bidirectional LSTM	64 units, dropout 0.3	Bidirectional sequence learning
Dense	32 units, ReLU, L2 reg	Feature extraction
BatchNormalization	-	Training stability
Dropout	Rate = 0.4	Prevent overfitting
Dense (output)	1 unit, Sigmoid	Binary classification

Total parameters: 2,088,769 (2,088,705 trainable, 64 non-trainable from BatchNorm). The extra non-trainable parameters are from the BatchNormalization layer.

5.2 Training Curves

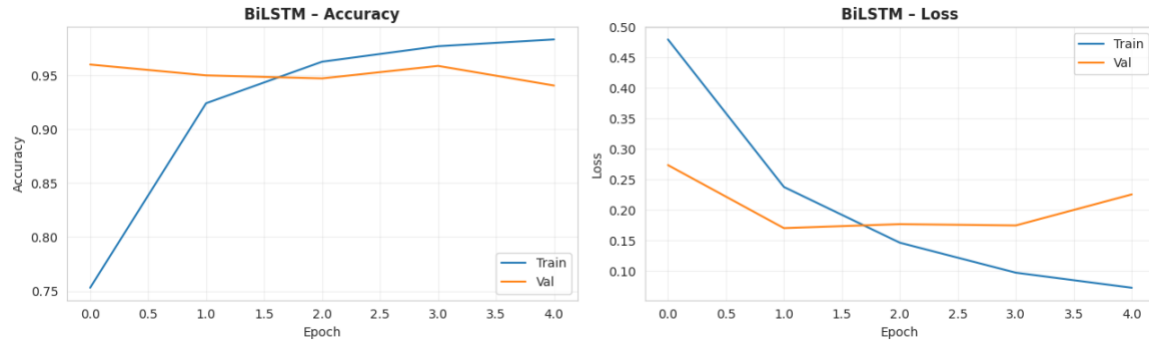


Figure 8 BiLSTM training and validation accuracy/loss over epochs

The BiLSTM trained for only 5 epochs before early stopping, restoring weights from epoch 2. This is quite fast in terms of epochs, though each epoch took much longer (about 68 seconds). The model reached high validation accuracy (0.9599) already in epoch 1, showing it learns quickly. However, it started overfitting from epoch 3 onwards.

5.3 Confusion Matrix



Figure 9 Confusion matrix for BiLSTM on test set (threshold = 0.807)

Metric	Value
Accuracy	0.9590
Precision (hate)	0.7541
Recall (hate)	0.6161
F1 (hate)	0.6781
Macro F1	0.8281
Best Threshold	0.807
Training Time	5.73 min

The BiLSTM scored a macro F1 of 0.8281, just slightly below the Simple RNN. It was actually very competitive, with slightly higher precision on hate (0.7541 vs 0.7527) but slightly lower recall. The model took much longer to train (5.73 minutes), making the Simple RNN a better choice in terms of efficiency.

6. Model 3 – Bidirectional LSTM with GloVe Embeddings

6.1 What are GloVe Embeddings?

GloVe (Global Vectors for Word Representation) is a set of pretrained word vectors. The specific vectors used here were "glove-twitter-100" from gensim – vectors trained on 2 billion tweets, giving each word a 100-dimensional vector that captures its meaning in Twitter-specific context.

This model used the same BiLSTM architecture as Model 2, but replaced the trainable embedding layer with a frozen GloVe embedding matrix. "Frozen" means the embedding weights were not updated during training. Out of 19,999 words in our vocabulary, 14,203 (71%) had a matching GloVe vector.

6.2 Training Curves

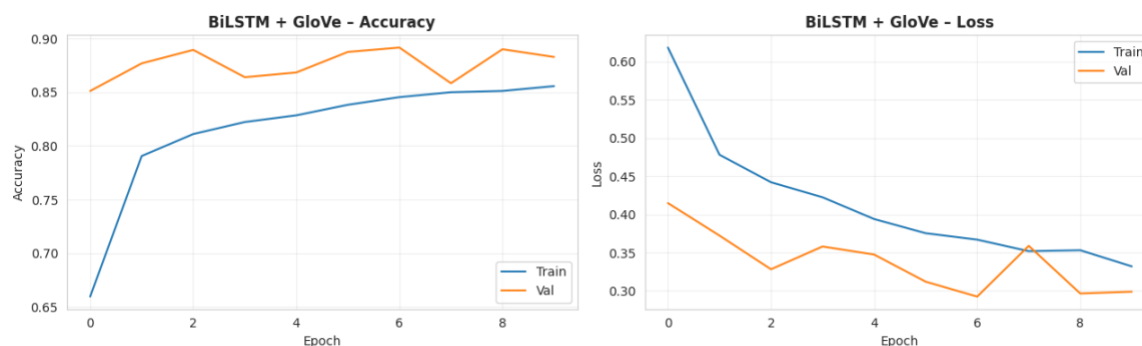


Figure 10 BiLSTM + GloVe training and validation accuracy/loss over epochs

The GloVe model trained for 10 epochs. Unlike the other two models, it was slower to learn – validation accuracy was only 0.85 in epoch 1, compared to 0.96 for the BiLSTM without GloVe. This is because the frozen embedding meant the model could not adapt the word vectors to this specific task. Training time was the longest at 9.69 minutes.

6.3 Confusion Matrix

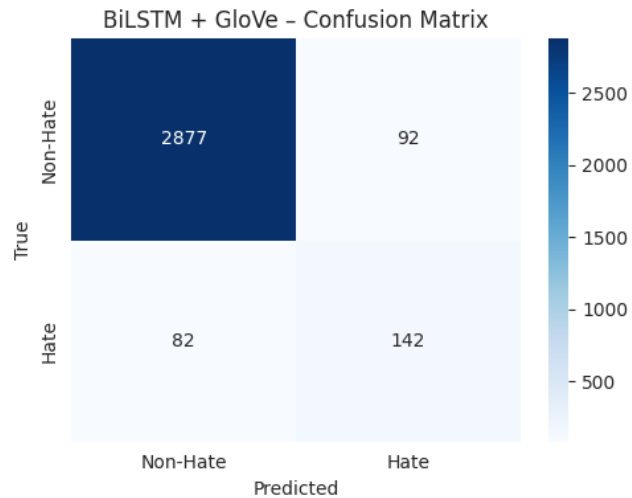


Figure 11 Confusion matrix for BiLSTM + GloVe on test set (threshold = 0.843)

Metric	Value
Accuracy	0.9455
Precision (hate)	0.6068
Recall (hate)	0.6339
F1 (hate)	0.6201
Macro F1	0.7954
Best Threshold	0.843
Training Time	9.69 min

The GloVe model had the lowest macro F1 (0.7954) but interestingly had slightly better recall for hate speech (0.6339 vs 0.6250 for RNN). This means it caught slightly more hate tweets, but with lower precision – it also incorrectly flagged more non-hate tweets as hate.

7. Overall Model Comparison

7.1 Validation Accuracy Over Epochs

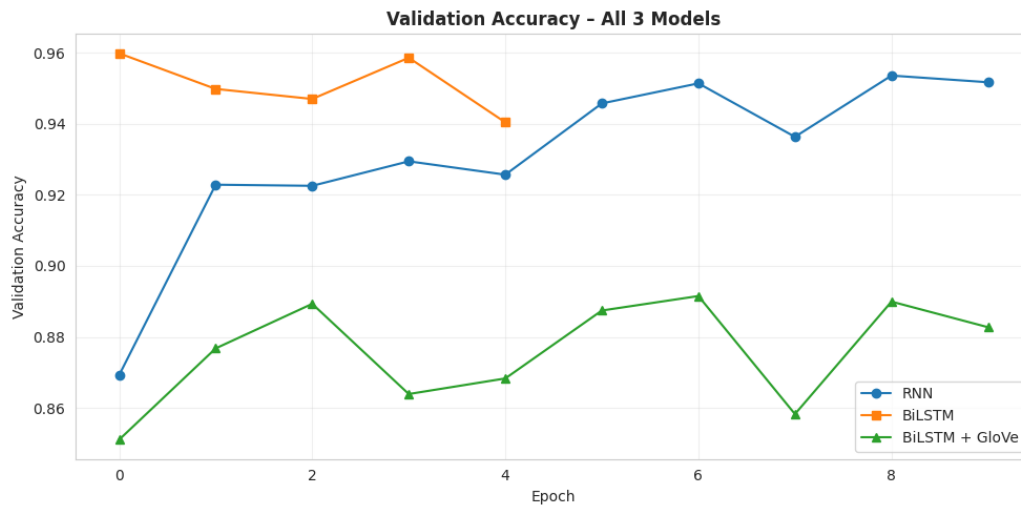


Figure 12 Validation accuracy for all three models over training epochs

7.2 Final Macro F1 Comparison

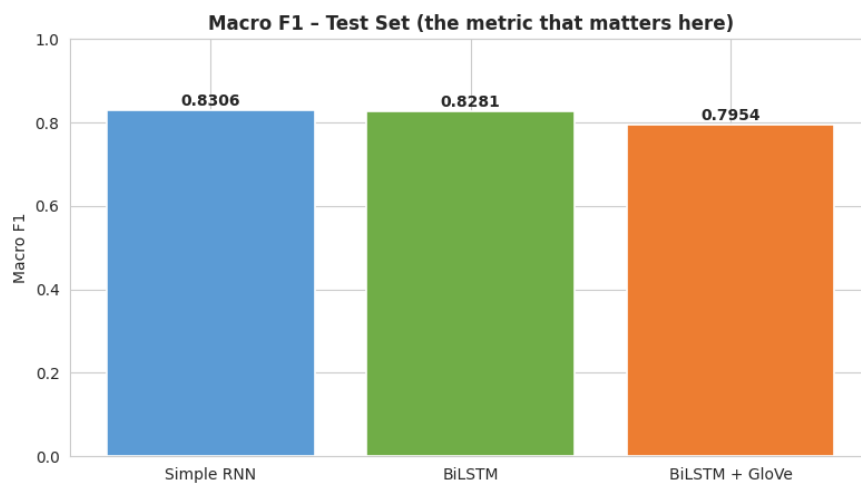


Figure 13 Macro F1 scores on test set for all three models

Model	Accuracy	Precision	Recall	Macro F1	Train Time	Epochs

Simple RNN		0.9593	0.7527	0.6250	0.8306	0.53 min	10
BiLSTM		0.9590	0.7541	0.6161	0.8281	5.73 min	5
BiLSTM + GloVe		0.9455	0.6068	0.6339	0.7954	9.69 min	10

Looking at the results, the Simple RNN actually came out on top with a macro F1 of 0.8306. The Bidirectional LSTM was very close at 0.8281 – only a 0.0025 difference – so in terms of predictive quality they are practically the same. Both trainable-embedding models significantly outperformed the frozen GloVe model.

The most surprising result was that the pretrained GloVe embeddings did not help. This is likely because the dataset is relatively small (~25,000 training examples), and when you freeze the embeddings you are limited to what GloVe already knows. The trainable models could adjust the embeddings specifically for this hate-detection task, which gave them an edge.

The Simple RNN was 10x faster than the BiLSTM and 18x faster than the GloVe model. For a task like this where speed matters (real-time tweet moderation), the RNN offers an excellent accuracy-efficiency trade-off.

8. Threshold Tuning

One important technique used in this project was threshold tuning. By default, neural networks with a sigmoid output use a 0.5 cut-off to decide between classes. But on heavily imbalanced data, this often gives poor results for the minority class.

The approach here was to sweep different threshold values on the validation set and pick the one that maximised the F1 score for the hate class. The chosen thresholds were then locked and used only once on the test set, keeping the evaluation clean and unbiased.

Model	Best Threshold	Val F1 at Best Threshold
Simple RNN	0.894	0.7100
BiLSTM	0.807	0.7169
BiLSTM + GloVe	0.843	0.6494

All three models required a threshold well above 0.5 – the RNN needed 0.894, meaning it needed to be almost 90% confident before labelling something as hate. This makes sense given the imbalance: the models are trained to be more conservative about predicting the rare class.

9. Error Analysis

The best model (Simple RNN) misclassified 130 out of 3,193 test tweets. Here are some examples of the errors:

Tweet (cleaned)	True Label	Predicted	Observation
ukraine mp amp kiev commander say deny deaths volunteer disrespect fall fighters	Non-Hate	Hate	War-related words triggered a false positive
tdc media amp must confuse course hate crime radicalislamicterrorist attack usa	Non-Hate	Hate	Contains "hate" and extremist terms but is reporting on them
democrats st tell joke	Hate	Non-Hate	Political sarcasm – hard to detect without context
white crowd turn back black basketball team high school game trump	Hate	Non-Hate	Coded language, not obviously hateful on surface
omg use word broad people agree trigger	Hate	Non-Hate	Low confidence (prob = 0.079), subtle context missed

The hardest cases for the model are tweets that discuss hate speech without being hateful themselves (false positives), and tweets that use coded or sarcastic language (false negatives). These are genuinely difficult even for humans to classify, and they represent the limits of surface-level text classification.

10. Challenges Faced

Several challenges came up during this project:

- Class imbalance: With a 13:1 ratio between non-hate and hate tweets, initial models simply predicted "non-hate" for everything. This needed class weighting and threshold tuning to fix.
- Choosing the right metric: Early on, accuracy looked great (over 90%) but was misleading. Switching to macro F1 gave a much more honest view of how the models were actually performing.
- Training time: The BiLSTM and GloVe models took several minutes per run. Running multiple experiments with different hyperparameters was time-consuming.
- GloVe download: The GloVe Twitter model is about 390MB. Downloading it in Colab each time was slow, and the first download had to be managed carefully.
- Sarcasm and coded language: Many hate tweets use indirect or coded language that is very difficult for the model to catch. This is a known limitation of current NLP approaches.
- Hyperparameter tuning: Choosing the right dropout rates, learning rates, and architecture details required trial and error. Some settings that worked well for the BiLSTM did not work as well for the RNN.

11. Conclusion

This project successfully built and compared three deep learning models for detecting hate speech in tweets. All three models were trained on an imbalanced binary classification dataset and evaluated using macro F1 as the primary metric.

The Simple RNN turned out to be the best model with a macro F1 of 0.8306 on the test set, slightly ahead of the BiLSTM (0.8281). The pretrained GloVe model came in third at 0.7954. One of the main takeaways is that pretrained embeddings do not always help, especially when they are frozen and the dataset is small enough that the model can learn good representations from scratch.

The most important lesson from this project was about dealing with imbalanced data. Without class weighting and threshold tuning, all models would have appeared to perform well based on accuracy alone, but would have been useless at actually catching hate speech. Using macro F1 and tuning the decision threshold on the validation set made a significant difference to the real-world utility of the models.

Future work could explore using a small Transformer model like DistilBERT, which would likely outperform all three models here. Collecting more hate speech examples or using data augmentation could also help the models generalise better to the minority class.